Faculty of Computing

Year 1 Semester 2 (2025)

SE1020 – Object Oriented Programming Lab Sheet 04

Question 1

1. Develop a system to manage book details for a Library Management System.

   Scenario:

   Imagine you are developing **a Library Management System** for a university. The system needs to handle different types of books efficiently.

   (a) Create a class called Book with the title (String), author (String), ISBN (String), and price (double) as attributes.

   (b) Implement constructor overloading with the following variations:

   I. A constructor that initializes only title and author.

   II. A constructor that initializes title, author, and ISBN.

   III. A constructor that initializes all attributes (title, author, ISBN, and price).

   (c) Implement method overloading for a method named **getBookDetails()** with the following variations:

   I. No parameters → Returns all details of the book.

   II. One parameter (ISBN) → Returns details of the book with the given ISBN.

   (d) Create a main method and create objects using the above constructors. Call the **getBookDetails** methods and display the results.

Question 02

You have been hired by an E-Commerce Company to develop a **Shopping Discount System** that calculates the final price of products based on discounts and promotional offers.

Each product has a base price, and customers can receive discounts based on their membership type or seasonal promotions. The system should allow for flexible discount calculations and ensure that all prices are correctly computed.

The discount policy is as follows:

Membership Type	Discount Percentage
Premium Member	20% Off
Regular Member	10% Off
Non-Member	No Discount (0%)

Additionally, the company offers seasonal promotions, which provide an extra discount on top of membership discounts.

a) Implement a Product class with four attributes: productID, productName, basePrice, and membershipType (Premium, Regular, or Non-Member).

b) Overload the default constructor of the Product class as follows:

   a. A constructor that accepts only productID and productName.
   b. A constructor that accepts productID, productName, and basePrice.
   c. A constructor that initializes all four attributes

c) Implement getters and setters for membershipType and basePrice. The setter for membershipType must ensure that only valid types (Premium, Regular, Non-Member) are accepted; otherwise, it should set to Non-Member.

d) Create two versions of the calculateFinalPrice() method as;

   a. A method that calculates the final price based on the membership discount.
   b. Another that allows an extra seasonal discount (amount
   c. as a parameter) without modifying the stored values.

e) Implement a displayProductDetails() method to print all relevant details, including the final price after discounts.

f) Implement a main method and create Product objects to demonstrate the above variations of constructors. Then call the appropriate methods to calculate the final prices and display all the details of each product.

[Hint: Refer to the given sample output below]

```
Product ID: P1001
Name: Laptop
Base Price: $1000.0
Membership Type: Premium Member
Final Price (after discount): $800.0

Product ID: P1002
Name: Smartphone
Base Price: $600.0
Membership Type: Regular Member
Final Price (after discount): $540.0

Product ID: P1003
Name: Headphones
Base Price: $200.0
Membership Type: Non-Member
Final Price (after discount): $200.0

Estimated price for Smartphone with extra 5$ seasonal discount: $535.0
```

Question 03

Please submit your answer to the question below to Git. When committing the file, ensure that you include a meaningful commit message (e.g., "Lab04 - Question03").

You have been hired by a Ride-Sharing Company to develop a **Driver Performance Management System** that tracks driver details, calculates earnings, and rewards performance - based bonuses. Each month, drivers receive a performance score (1 to 5) based on customer ratings, with higher scores leading to higher bonus percentages of their base salary.

The bonus policy is as follows:

Performance Score	Bonus Percentage
5	25% of Base Salary
4	20% of Base Salary
3	15% of Base Salary
2	10% of Base Salary
1	No Bonus (0%)

a) Implement a class called **Driver** that includes four attributes: driverID, driverName, baseSalary, and performanceScore.

b) The class should support constructor overloading, allowing different ways to initialize a Driver object. There should be three constructors:

- one that accepts only driverID and driverName
- another that includes only the baseSalary
- third that initializes all four attributes, including performanceScore.

c) Implement getters and setters for driverID and performanceScore. The setter for performanceScore must validate that the value is between 1 and 5. If an invalid score is provided, it should default to 1.

d) Create two versions of the calculateBonus() method:

- one that calculates the bonus based on the driver's actual performance score (attribute)
- another that allows a custom score (parameter) to be used without modifying the stored value.

e) Additionally, implement a calculateTotalSalary() method to compute the final salary by adding the base salary and the calculated bonus.

f) Implement a main method and create Driver objects to demonstrate the above variations of constructors. Then call the appropriate methods to calculate the salaries and display all the details of each driver.

[**Hint: Refer to the given sample output below**]

```
Driver ID: D1001
Name: John Smith
Base Salary: 4000.0
Performance Score: 5
Bonus: 1000.0
Total Salary: 5000.0

Driver ID: D1002
Name: Emily Davis
Base Salary: 3500.0
Performance Score: 3
Bonus: 525.0
Total Salary: 4025.0

Driver ID: D1003
Name: Michael Johnson
Base Salary: 5000.0
Performance Score: 1
Bonus: 0.0
Total Salary: 5000.0

Bonus for driver3 with custom rating 4: 1000.0
```